6. Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.
Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.
code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define SERVER_FIFO "server_fifo"
#define CLIENT_FIFO "client_fifo"

int main()
{
   char message[100];
   char response[100];

   // Create named pipes
   mkfifo(SERVER_FIFO, 0666);
   mkfifo(CLIENT_FIFO, 0666);

   printf("Server started. Waiting for client...\n");

   while (1)
   {
      // Open server FIFO for reading
```

```
    int fd1 = open(SERVER_FIFO, O_RDONLY);

    read(fd1, message, sizeof(message));
    close(fd1);

    printf("Client: %s\n", message);

    // Process the message
    if (strcmp(message, "hello") == 0)
        strcpy(response, "Hello from server!");
    else if (strcmp(message, "exit") == 0)
    {
        strcpy(response, "Server shutting down.");

        int fd2 = open(CLIENT_FIFO, O_WRONLY);
        write(fd2, response, strlen(response) + 1);
        close(fd2);

        break;
    }
    else
        strcpy(response, "Message received and processed.");

    // Send response
    int fd2 = open(CLIENT_FIFO, O_WRONLY);

    write(fd2, response, strlen(response) + 1);

    close(fd2);
}

unlink(SERVER_FIFO);
```

```
    unlink(CLIENT_FIFO);

    return 0;
}
```

Part 2 :client

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define SERVER_FIFO "server_fifo"
#define CLIENT_FIFO "client_fifo"

int main()
{
    char message[100];
    char response[100];

    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);

    // Remove newline
    message[strcspn(message, "\n")] = '\0';

    // Send message to server
    int fd1 = open(SERVER_FIFO, O_WRONLY);

    write(fd1, message, strlen(message) + 1);

    close(fd1);
```

```
    // Receive response from server
    int fd2 = open(CLIENT_FIFO, O_RDONLY);

    read(fd2, response, sizeof(response));

    close(fd2);

    printf("Server: %s\n", response);

    return 0;
}
```

Output:

```
Terminal 1:
pranav@Pranavs-MacBook-Air dir1 % vi week6.c
pranav@Pranavs-MacBook-Air dir1 %  gcc week6.c -o week6
pranav@Pranavs-MacBook-Air dir1 % ./week6
Server started. Waiting for client...
Terminal 2:
pranav@Pranavs-MacBook-Air ~ % cd dir1
pranav@Pranavs-MacBook-Air dir1 % vi week6part2.c
pranav@Pranavs-MacBook-Air dir1 %  gcc week6part2.c -o
week6part2
pranav@Pranavs-MacBook-Air dir1 % ./week6part2
Enter message: hello darkness my old friend
Server: Message received and processed.
Terminal 1:
Client: hello darkness my old friend
```